Vulnlog User Guide

Table of Contents

1. What is Vulnlog? . . . 2
   1.1. Features . . . 2
   1.2. When to Use Vulnlog . . . 3
   1.3. How Vulnlog Compares . . . 3
      1.3.1. vs. Scanner-native suppression files . . . 4
      1.3.2. vs. Scanner vendor UIs . . . 4
      1.3.3. vs. Ticketing-System, Spreadsheets and Confluence pages . . . 4
   1.4. How Vulnlog Relates to Other Standards . . . 4
   1.5. Supported Reporters . . . 5
2. Getting Started . . . 5
   2.1. Creating a Vulnlog File . . . 5
   2.2. A Minimal Example . . . 5
   2.3. IDE Setup . . . 6
   2.4. CI Pipeline Integration . . . 6
3. Common Workflows . . . 7
   3.1. New finding — triage later . . . 7
   3.2. Finding does not affect us . . . 7
   3.3. Fixed in the next release . . . 8
   3.4. Risk accepted with temporary suppression . . . 8
   3.5. Internal dependency, no customer impact . . . 9
   3.6. Non-scanner source . . . 9
4. Managing Multiple Release Branches . . . 10
5. CLI Reference . . . 10
   5.1. Usage . . . 10
   5.2. Global Flags . . . 10
   5.3. Filtering . . . 10
   5.4. Commands . . . 11
      5.4.1. `vulnlog init` . . . 11
      5.4.2. `vulnlog validate` . . . 11
      5.4.3. `vulnlog lint` . . . 11
      5.4.4. `vulnlog stats` . . . 11
      5.4.5. `vulnlog report` . . . 11
      5.4.6. `vulnlog suppress` . . . 11
      5.4.7. `vulnlog vex` . . . 11
   5.5. Exit Codes . . . 12
6. Full Example . . . 12

1. What is Vulnlog?

Vulnlog is a tool for tracking, analyzing, and managing software vulnerabilities reported by SCA (Software Composition Analysis) scanners.

It combines a simple YAML file format with a CLI that generates suppression files, HTML reports, and OpenVEX statements. The name combines "vulnerability" and "log" — Vulnlog is a persistent, version-controlled record of your team's vulnerability analysis.

For the full YAML format reference, see the Vulnlog Specification.

1.1. Features

Vulnerability tracking

Record and manage vulnerability findings from multiple SCA scanners in a single YAML file. Each finding captures the scanner report, affected packages, and the team's triage verdict.

Verdict-driven workflow

Classify vulnerabilities with a clear verdict: `affected`, `not_affected`, `fixed`, or `risk_acceptable`. Entries without a verdict are tracked as under investigation. Each verdict type enforces appropriate companion fields (justification, severity, fix reference).

Flexible tagging

Tag vulnerability entries with a team-defined taxonomy. Tags can represent deliverables (`binary`, `container`), usage contexts (`build-dep`, `test-infra`), or any other categorization. Use tags to filter CLI output and scope VEX products.

Suppression file generation

Generate scanner-specific suppression files (`.trivyignore`, `.snyk`, etc.) from the Vulnlog file. Only suppress what needs to be suppressed: Vulnerabilities marked as fixed are not suppressed any longer to assert the fix was successful.

HTML report generation

Generate self-contained HTML reports for communicating vulnerability status to team members, partners, and customers. Reports include summary statistics, filtering, sorting, and search by any identifier.

OpenVEX statement generation

Generate OpenVEX documents per release for standards-compliant vulnerability communication. Verdicts map directly to OpenVEX statuses. Tags scope VEX products to specific deliverables.

Multi-scanner support

Track findings from multiple SCA scanners in a single file. Switch scanners without losing analysis history.

Non-scanner sources

Record vulnerabilities from vendor advisories (e.g., Oracle CPU), penetration tests, and other non-scanner sources alongside scanner findings.

IDE integration

JSON Schema published to SchemaStore provides validation, autocompletion, and inline documentation.

Git-native

Vulnlog files are plain YAML designed for version control. Changes are tracked via Git history. Each release branch has its own file, which archives naturally at end of life.

Validation and linting

Validate Vulnlog files against structural rules and cross-reference integrity. Lint for consistent field ordering and formatting.

CI/CD integration

The CLI runs on developer machines and in CI pipelines. Generate suppression files in CI to keep scanner pipelines green for triaged findings. Generate reports and VEX documents as build artifacts.

1.2. When to Use Vulnlog

Vulnlog is most valuable when your team:

- Uses multiple SCA scanners (Trivy, Snyk, Grype, OWASP Dependency Check, etc.) and needs a unified view of findings across scanners.
- Maintains multiple release branches and needs to track vulnerability decisions per branch without duplicating work.
- Needs to communicate vulnerability status to customers, partners, or auditors via HTML reports or VEX statements.
- Wants to keep vulnerability analysis in the repository, next to the code, rather than in external tools.
- Wants to prevent stale suppression files that shadow real findings after vulnerabilities have been fixed.
- Do not want to lose vulnerability analysis, suppressions or VEX statements when moving to another SCA tool.

Vulnlog may be more than you need if you have a single scanner on a single branch — in that case, the scanner's native suppression mechanism may suffice.

1.3. How Vulnlog Compares

1.3.1. vs. Scanner-native suppression files

Scanner-native suppression files (`.trivyignore`, `.snyk`) track **what to suppress** but not **why**. They don't record the analysis, the verdict reasoning, or who made the decision. When team members rotate, the reasoning is lost. Vulnlog generates these files from a richer decision record, keeping the **why** alongside the **what**.

Suppression files also tend to accumulate stale entries for vulnerabilities that were fixed long ago. Vulnlog never generates suppressions for fixed vulnerabilities, keeping the suppression file accurate.

1.3.2. vs. Scanner vendor UIs

Scanner vendor UIs (Snyk dashboard, GitHub Security tab or Semgrep UI) provide good visibility into findings for one scanner. But they create vendor lock-in: your triage decisions and analysis live in the vendor's platform. Switching scanners means starting over. Vulnlog keeps your analysis in a YAML file you own, and generates outputs for any supported scanner.

1.3.3. vs. Ticketing-System, Spreadsheets and Confluence pages

Spreadsheets and wiki pages are flexible but disconnected from the code. They require manual synchronization, don't support automated output generation, and are easy to forget when the code changes. Vulnlog lives in the repository and generates suppression files, reports, and VEX documents directly from the data.

1.4. How Vulnlog Relates to Other Standards

VEX (Vulnerability Exploitability eXchange)

Vulnlog generates OpenVEX documents. VEX communicates whether vulnerabilities are exploitable in the context of a specific product. Vulnlog is the decision record; VEX is one of its outputs.

SBOM (Software Bill of Materials)

Complementary. SBOMs describe what components are in your software. Vulnlog tracks decisions about vulnerability findings in those components. Together they answer "what do we have?" (SBOM) and "what have we decided about findings?" (Vulnlog).

SARIF (Static Analysis Results Interchange Format)

A future Vulnlog output. SARIF provides a standardized format for findings that integrates with development tools (GitHub Code Scanning, Azure DevOps, VS Code).

CSAF (Common Security Advisory Framework)

Complementary. CSAF is a broader advisory format. Vulnlog focuses specifically on team-level triage decisions and suppression management.

1.5. Supported Reporters

Type	Scanner / Source	Suppression Support
`trivy`	Trivy	Yes
`snyk`	Snyk	Yes
`dependency_check`	OWASP Dependency Check	Yes
`grype`	Grype	Planned
`rust_audit`	Cargo Audit (Rust)	Planned
`semgrep`	Semgrep	No
`npm_audit`	npm audit	No
`github_advisory`	GitHub Advisory Database	No
`other`	Generic / non-scanner source (e.g., Oracle CPU, penetration test reports)	No

2. Getting Started

2.1. Creating a Vulnlog File

Use the CLI to scaffold a new project:

```
vulnlog init --project "My App" --organization "Acme Corp" -o my-app.vl.yaml
```

This creates a minimal Vulnlog file with only the required sections.

NOTE

File Name Convention

For simple projects that only work with one Vulnlog file, `vulnlog.[yaml|yml]` are valid file names.

If you need to work with multiple Vulnlog files, such as when working with multiple release branches or products, use the file name convention `<project-name>.vl.[yaml|yml]`.

2.2. A Minimal Example

```
schemaVersion: "1"

project:
  organization: Acme Corp
  name: Acme Web App
  author:  Acme Corp Security Team
```

```
releases:
  - id: 1.0.0
    published_at: 2026-01-15

vulnerabilities:

  - id: CVE-2026-1234
    releases: [ 1.0.0 ]
    description: Remote code execution in example-lib
    packages: [ "pkg:npm/example-lib@2.3.0" ]
    reports:
      - reporter: trivy
    analysis: >
      The vulnerable code path is not reachable in our application
      because we only use the safe subset of the API.
    verdict: not_affected
    justification: vulnerable_code_not_in_execute_path
```

2.3. IDE Setup

Vulnlog's JSON Schema is published to SchemaStore, which means IDEs with YAML support can automatically provide validation and autocompletion for `.vl.[yaml|yml]` and `vulnlog.[yaml|yml]` files.

2.4. CI Pipeline Integration

A typical CI workflow with Vulnlog:

1. The SCA scanner runs and finds vulnerabilities.
2. Without Vulnlog, the pipeline fails on known, already-triaged findings.
3. With Vulnlog, the pipeline runs `vulnlog suppress` before the scanner to generate up-to-date suppression files. The scanner skips triaged findings and only fails on genuinely new ones.

Example GitHub Actions workflow:

```
jobs:
  security-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install Vulnlog CLI
        run: # install step

      - name: Generate Trivy suppression file
        run: vulnlog suppress my-app.vl.yaml --reporter trivy --release ${{
github.ref_name }}
```

```
      - name: Run Trivy scan
        uses: aquasecurity/trivy-action@master
        with:
          scan-type: fs
          trivyignores: .trivyignore

      - name: Generate vulnerability report
        if: always()
        run: vulnlog report my-app.vl.yaml --release ${{ github.ref_name }} -o
vulnlog-report.html

      - name: Upload report
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: vulnerability-report
          path: vulnlog-report.html
```

3. Common Workflows

3.1. New finding — triage later

Create a minimal stub entry:

```
  - id: CVE-2026-5678
    releases: [ 2.0.0 ]
    packages:
      - "pkg:npm/image-lib@3.1.0"
    reports:
      - reporter: trivy
        at: 2026-03-01
    tags: [ webapp ]
```

3.2. Finding does not affect us

```
  - id: CVE-2026-5678
    releases: [ 2.0.0 ]
    description: Memory corruption in image-lib
    packages:
      - "pkg:npm/image-lib@3.1.0"
    reports:
      - reporter: trivy
        at: 2026-03-01
    tags: [ webapp ]
    analysis: >
      The vulnerable code path requires SVG input processing.
```

```
    Our application only accepts PNG and JPEG formats.
  analyzed_at: 2026-03-02
  verdict: not_affected
  justification: vulnerable_code_not_in_execute_path
```

3.3. Fixed in the next release

```
- id: CVE-2026-5678
  releases: [ 2.0.0 ]
  description: Memory corruption in image-lib
  packages:
    - "pkg:npm/image-lib@3.1.0"
  reports:
    - reporter: trivy
      at: 2026-03-01
  tags: [ webapp ]
  analysis: >
    Confirmed exploitable via crafted PNG uploads.
    Updated image-lib to 3.2.0 which contains the fix.
  analyzed_at: 2026-03-02
  severity: high
  verdict: fixed
  fixed_in: 2.0.1
  fixed_at: 2026-03-03
  fix_ref: "https://jira.example.com/browse/SEC-99"
  fix_note: "Updated image-lib from 3.1.0 to 3.2.0"
```

3.4. Risk accepted with temporary suppression

```
- id: CVE-2026-5678
  releases: [ 2.0.0 ]
  description: Memory corruption in image-lib
  packages:
    - "pkg:npm/image-lib@3.1.0"
  reports:
    - reporter: trivy
      at: 2026-03-01
      suppress:
        expires_at: 2026-03-08
  tags: [ webapp ]
  analysis: >
    Vulnerability confirmed but only exploitable with
    specially crafted input that our validation layer rejects.
    Accepting risk until upstream releases a stable fix.
  analyzed_at: 2026-03-02
  severity: medium
  verdict: risk_acceptable
```

```
    comment: >
      Risk accepted by project lead. Revisit when upstream
      releases a stable fix.
```

3.5. Internal dependency, no customer impact

```
  - id: CVE-2026-9999
    releases: [ 2.0.0 ]
    description: Command injection in build-tool
    packages:
      - "pkg:npm/build-tool@5.0.0"
    reports:
      - reporter: trivy
        at: 2026-03-01
    tags: [ build-dep ]
    analysis: >
      Affects the build pipeline only. The vulnerable command
      injection requires untrusted input which the CI environment
      does not expose.
    analyzed_at: 2026-03-01
    verdict: not_affected
    justification: vulnerable_code_cannot_be_controlled_by_adversary
```

3.6. Non-scanner source

```
  - id: CVE-2022-21449
    name: Psychic Signatures
    releases: [ 8.1.2 ]
    description: ECDSA signature verification bypass in Java SE
    packages:
      - "pkg:maven/com.oracle/jdk@17.0.2"
    reports:
      - reporter: other
        at: 2022-04-19
        source: "Oracle Critical Patch Update April 2022"
    tags: [ binary, container ]
    analysis: >
      The project uses Java 17 and ECDSA signatures for
      JWT token verification. Confirmed exploitable.
    analyzed_at: 2022-04-20
    severity: critical
    verdict: fixed
    fixed_in: 8.1.2
    fixed_at: 2022-04-22
```

4. Managing Multiple Release Branches

Maintain one Vulnlog file per release branch, all on the main branch:

```
my-app-8.1.vl.yaml    # Release branch 8.1.x
my-app-8.2.vl.yaml    # Release branch 8.2.x
my-app-8.3.vl.yaml    # Release branch 8.3.x
```

This avoids cherry-picking vulnerability analysis to older branches. When a release branch reaches end of life (EoL), its Vulnlog file naturally becomes a historical archive.

Alternatively, maintain a Vulnlog file directly on each release branch. Both approaches can coexist.

5. CLI Reference

5.1. Usage

```
vulnlog <command> <file> [flags]
```

Every command except `init` requires a Vulnlog file as a positional argument.

5.2. Global Flags

Flag	Description
`-q, --quiet`	Suppress non-essential output.
`-v, --verbose`	Verbose output.
`--no-color`	Disable colored output.
`--format <f>`	Output format: `yaml` (default). Future: `json`, `text`.
`--version`	Print version.
`--help`	Print help.

5.3. Filtering

Flag	Description
`--release <id>`	Include vulnerabilities affecting all releases up to and including this release.
`--tags <id>`	Filter by tag (repeatable for multiple tags).

5.4. Commands

5.4.1. `vulnlog init`

```
vulnlog init -o my-app.vl.yaml --project "My App" --organization "Acme Corp"
```

5.4.2. `vulnlog validate`

```
vulnlog validate my-app.vl.yaml
vulnlog validate my-app.vl.yaml --strict
```

5.4.3. `vulnlog lint`

```
vulnlog lint my-app.vl.yaml          # Check only
vulnlog lint my-app.vl.yaml --fix    # Rewrite with corrected ordering
```

5.4.4. `vulnlog stats`

```
vulnlog stats my-app.vl.yaml
vulnlog stats my-app.vl.yaml --release 8.1.1
```

5.4.5. `vulnlog report`

```
vulnlog report my-app.vl.yaml
vulnlog report my-app.vl.yaml --release 8.1.1 -o report.html
vulnlog report my-app.vl.yaml --tags container --release 8.1.1
```

5.4.6. `vulnlog suppress`

```
# Suppression for a specific reporter
vulnlog suppress my-app.vl.yaml --reporter trivy --release 8.1.1

# All reporters, all releases
vulnlog suppress my-app.vl.yaml

# Custom output path
vulnlog suppress my-app.vl.yaml --reporter trivy -o custom/.trivyignore
```

5.4.7. `vulnlog vex`

```
vulnlog vex my-app.vl.yaml --release 8.1.1
vulnlog vex my-app.vl.yaml --release 8.1.1 --tags binary -o vex-binary.json
```

5.5. Exit Codes

Code	Name	Description
`0`	Success	Command completed successfully.
`1`	General error	Unexpected error.
`2`	Validation error	The Vulnlog file contains validation errors.
`3`	File not found	The specified file does not exist.
`4`	Lint violation	The file has lint violations (without `--fix`).
`5`	Missing a required flag	A required flag was not provided.
`6`	Invalid flag value	A flag references an unknown entity or unsupported type.

6. Full Example

```yaml
schemaVersion: "1"

project:
  organization: Demo Project Inc
  name: Demo Project
  author: Demo Security Team
  contact: security@demo.com

tags:
  - id: binary
    description: Demo Project binary file
  - id: container
    description: Demo Project container image
  - id: sdk
    description: Demo Project SDK
  - id: build-dep
    description: Internal build dependency

releases:
  - id: 8.1.0
    published_at: 2021-01-15
    purls:
      - purl: "pkg:generic/demo-project@8.1.0"
        tags: [ binary ]
      - purl: "pkg:docker/demo-project@8.1.0"
        tags: [ container ]
```

```
  - id: 8.1.1
    published_at: 2021-02-11
    purls:
      - purl: "pkg:generic/demo-project@8.1.1"
        tags: [ binary ]
      - purl: "pkg:docker/demo-project@8.1.1"
        tags: [ container ]
  - id: 8.1.2

vulnerabilities:

  # No verdict - under investigation, with temporary suppression
  - id: CVE-2020-099
    releases: [ 8.1.0 ]
    description: Path traversal vulnerability in random-dependency
    packages: [ "pkg:maven/org.random/random-core@1.33.7" ]
    reports:
      - reporter: trivy
        at: 2020-12-28
        suppress:
          expires_at: 2021-01-02
    tags: [ build-dep ]
    analysis: >
      Vulnerability in internal build dependency.
      No fixed library available at the moment. Re-check later.
    comment: >
      Checked upstream issue tracker - fix expected in random-core 1.33.8.
      Monitor https://github.com/random/random-core/issues/421
    severity: low

  # Stub entry - quick triage, no description, no verdict yet
  - id: CVE-2021-007
    releases: [ 8.1.1 ]
    packages: [ "pkg:maven/com.example/some-lib@4.0.0" ]
    reports:
      - reporter: trivy
        at: 2021-03-10
    tags: [ binary ]

  # Verdict: not_affected - vulnerability does not impact this project
  - id: CVE-2021-002
    aliases: [ GHSA-xxxx-yyyy ]
    releases: [ 8.1.1 ]
    description: Deserialization vulnerability in some-dependency
    packages: [ "pkg:maven/example.org/some-dependency@1.1.2" ]
    reports:
      - reporter: trivy
        at: 2021-03-03
      - reporter: snyk
        at: 2021-03-03
        vuln_id: [ SNYK-2021-002 ]
```

```
    tags: [ binary, container ]
    analysis: >
      Demo project does not utilize some-dependency
      and therefore is not affected.
    analyzed_at: 2021-03-03
    verdict: not_affected
    justification: vulnerable_code_not_present

  # Verdict: fixed - resolved in a subsequent release (named vulnerability)
  - id: CVE-2021-44228
    name: Log4Shell
    aliases: [ GHSA-jfh8-c2jp-5v3q ]
    releases: [ 8.1.1 ]
    description: RCE vulnerability in Log4J via JNDI lookup
    packages: [ "pkg:maven/org.apache.logging.log4j/log4j-core@2.14.1" ]
    reports:
      - reporter: trivy
        at: 2021-12-10
    tags: [ binary ]
    analysis: >
      Demo project utilizes Log4J and a PoC showed that the project is vulnerable to
this RCE vulnerability.
    analyzed_at: 2021-12-10
    comment: >
      PoC exploit confirmed by Jane (security team) on 2021-12-10.
      See internal report attached to SEC-42.
    severity: critical
    verdict: fixed
    fixed_in: 8.1.2
    fixed_at: 2021-12-12
    fix_ref: "https://jira.example.com/browse/SEC-42"
    fix_note: "Updated Log4J from 2.14.1 to 2.17.1"

  # Verdict: not_affected - OS-level vulnerability in container base image
  - id: CVE-2024-5535
    releases: [ 8.1.1 ]
    description: Buffer overread in OpenSSL SSL_select_next_proto
    packages: [ "pkg:apk/alpine/openssl@3.1.4-r0" ]
    reports:
      - reporter: trivy
        at: 2024-06-28
    tags: [ container ]
    analysis: >
      Vulnerable OpenSSL is present in the container base image but the affected
NPN/ALPN negotiation code path is never invoked by our application.
    analyzed_at: 2024-06-29
    verdict: not_affected
    justification: vulnerable_code_not_in_execute_path

  # Verdict: risk_acceptable - vulnerability present but risk accepted
  - id: SNYK-2021-003
```

```
    releases: [ 8.1.2 ]
    description: DoS vulnerability in some-other-dependency
    packages: [ "pkg:maven/com.some.other/some-other-dependency@3.11.0" ]
    reports:
      - reporter: snyk
        at: 2022-04-28
    tags: [ binary, container ]
    analysis: >
      Demo project utilizes some-other-dependency but only in rare cases.
      Therefore the risk is considered low and a fix would potentially
      be breaking for customer setups.
    analyzed_at: 2022-04-29
    comment: >
      Risk accepted by project lead (decision documented in RISK-12).
      Revisit when some-other-dependency 4.0 is released.
    severity: low
    verdict: risk_acceptable

  # Report from a non-scanner source (Oracle CPU)
  - id: CVE-2022-21449
    name: Psychic Signatures
    releases: [ 8.1.2 ]
    description: ECDSA signature verification bypass in Java SE
    packages: [ "pkg:maven/com.oracle/jdk@17.0.2" ]
    reports:
      - reporter: other
        at: 2022-04-19
        source: "Oracle Critical Patch Update April 2022"
    tags: [ binary, container ]
    analysis: The project uses Java 17 and ECDSA signatures for JWT token
verification. Confirmed exploitable.
    analyzed_at: 2022-04-20
    severity: critical
    verdict: fixed
    fixed_in: 8.1.2
    fixed_at: 2022-04-22
    fix_ref: "https://jira.example.com/browse/SEC-55"
    fix_note: "Updated JDK from 17.0.2 to 17.0.3"
```